Appello 16 Settembre 2016



Politecnico di Milano

Anno accademico 2015-2016

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Ghezzi	☐ Mottola	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta l'impossibilità a partecipare all'appello successivo.
6. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 5:

Esercizio 1

La classe Borsa realizza una collezione i cui elementi sono esclusivamente numeri interi nell'intervallo 0...MAX, con MAX una costante non negativa e stabilita al momento della costruzione. Gli elementi posso essere ripetuti con cardinalita' arbitraria.

La seguente specifica descrive in modo informale il comportamento della classe. *Non vi sono altre eccezioni oltre a quelle segnalate nel codice*.

```
public class Borsa {
//overview: Una collezione non ordinata di elementi
//il cui valore \'e limitato fra 0 e MAX.

//la costante MAX:
public final int MAX;
//costruisce una Borsa vuota con elementi fino a max>0
public Borsa(int max)

//inserisce un elemento
public void insert(int x) throws OutofRangeException

//rimuove una comparsa di un elemento e
//restituisce true se una comparsa dell'elemento
//\'e ancora presente dopo l'eliminazione,
//false altrimenti
public boolean remove(int x)

//restituisce il numero di comparse dell'elemento i
public int card(int i)

//restituisce il numero di elementi distinti nella borsa
public int distinct()
}
```

1. Si indichi quali metodi della classe Borsa sono puri. Scrivere inoltre la specifica JML per il costruttore e i metodi insert e remove della classe Borsa, eventualmente utilizzando un opportuno oggetto astratto tipico se ritenuto conveniente.
2. Scrivere in JML l'invariante pubblico della classe e argomentare che è verificato dalla specifica da voi definita.

   Soluzione

   La specifica di tutti i metodi descrive l'effetto della chamata mostrando come cambiano i valori di `card(i)`. Esplicitiamo, pur se non richiesto, anche la specifica di `distinct()`, che è semplicemente derivata da `card(i)`.

   ```
   public class Borsa {
   //overview: Una collezione non ordinata di elementi
   //il cui valore \'e limitato fra 0 e MAX.

   //la costante MAX:
   public final int MAX;
   //@requires max>=0;
   //@ensures MAX==max &&
   //@  (\forall int i; 0<=i&&i<= MAX; card(i)==0);
   public Borsa(int max)
   ```

```
//@ensures 0<= x && x<= MAX &&
//@  (\forall int i; 0<=i && i<=MAX; card(x)==\old(card(x)) + (i==x ?  1 : 0);
//@signals (OutofRangeException e) !(0<= x && x<= MAX) &&
//@  (\forall int i; 0<=i&&i<= MAX; card(i)==\old(card(i)));
public void insert(int x) throws OutofRangeException

//@ensures \result == (x>0 && x<=MAX && card(x)>0) &&
//@   (\forall int i; 0<=i&&i<= MAX;
//@        card(i)== \old(card(x)) - (i==x&&\old(card(x)>0 ?  1 : 0)));
public boolean remove(int x)

//restituisce il numero di comparse dell'elemento i
public /*@ pure @*/ int card(int i)

//@ensures \result == (\num_of int i; 0<=i && i<=MAX; card(i)>0);
public /*@ pure @*/  int distinct()
}

public invariant MAX>=0 && distinct()>=0   &&
  (\forall int i; i<0 || i>MAX; card(i)==0) &&
  (\forall int i; 0<=i && i<=MAX; card(i)>=0);
```

Esercizio 2

Nell'implementazione della classe Borsa dell'esercizio 1, si decide di rappresentare la Borsa come un array elems di dimensioni MAX+1, L'intero contenuto in posizione i di elems rappresenta il numero di comparse dell'elemento i nella Borsa. Un intero numDistinti memorizza quanti elementi distinti sono presenti nell'array (ossia proprio il valore restituito dal metodo distinct).

```
//rep:
private int[] elems;
private int numDistinti;
```

Si definiscano in JML l'invariante di rappresentazione e la funzione di astrazione. Inoltre, si implementi il metodo remove nella maniera ritenuta piu' opportuna.

Soluzione

Funzione di astrazione:

```
(\forall int i; 0<=i && i<=MAX; card(i)==elems[i];) &&
numDistinti == distinct();
```

L'invariante di rappr. afferma che la lunghezza di elems è uguale a MAX+1, che elems contiene sono valori nonnegativi e stabilisce infine che numDistinti contiene effettivamente il numero di valori distinti in elems:

```
private invariant MAX+1==elems.length &&
  (\forall int i; 0<=i && i<=MAX; elems[i]>=0;) &&
  numDistinti == (\num_of int j; 0<=j & j<=MAX; elems[j]>0);
```

Una semplice implementazione, basata sul RI, è:

```
public boolean remove(int x) {
  if (x<0||x>MAX || elems[x]==0)
     return false;
 elems[x]--;
 if (elems[x]==0) {
     numDistinti--;
     return false;
 }
 return true
}
```

Esercizio 3

Si consideri la seguente definizione di una classe Person:

```
public class Person {
  private final String name;
  private final int age;
  public Person(final String theName, final int theAge) {
    name = theName;
    age = theAge;
  }
  public int ageDifference(final Person other) {
    return age - other.age;
 }
  public String toString() {
    return String.format("%s - %d", name, age);
  }
...
}
```

Il seguente frammento di codice definisce un ArrayList people di Person e, a mero titolo di esempio, lo inizializza con alcuni valori.

```
final List<Person> people = Arrays.asList(
  new Person("John", 20),
  new Person("Sara", 21),
  new Person("Jane", 21),
  new Person("Greg", 35));
```

Si rammenta qui la definizione in Java 8 del notissimo interfaccia Comnparator:

```
@FunctionalInterface
public interface Comparator<T> {
  // \result<0 se o1 minore di o2, =0 se uguali, altrimenti >0
  int compare(T o1, T o2)
  ...
}
```

Scrivere un frammento di programma Java 8 in stile funzionale che stampa le persone contenute in people in ordine di eta' decrescente (nell'esempio, Greg, Sara, Jane, John).

Si suggerisce di utilizzare l'operazione `sorted()` definita per tutti gli stream, che prevede un argomento funzionale che sia un Comparator. Ad esempio, se la variabile stringCollection è una List di oggetti String (la cui classe fornisce un metodo compareTo), il seguente frammento definisce uno stream ordinato in ordine crescente.

```
stringCollection
    .stream()
    .sorted((a, b) -> b.compareTo(a))
```

Soluzione:

```
  people.stream()
      .sorted((person1, person2) -> person2.ageDifference(person1))
      .forEach(System.out::println);
```

Si noti che la lambda `(person1, person2) -> person2.ageDifference(person1)` definisce una funzione anonima che rispetta l'interfaccia Comparator.

E' possibile scrivere una versione ancora piu' compatta:

```
people.stream()
        .sorted(Person::ageDifference)
        .forEach(System.out::println);
```

Esercizio 4

Si deve progettare un sistema per la gestione in tempo reale dei tempi di arrivo di autobus urbani presso le fermate della linea che percorrono. Una linea include almeno una fermata oltre al capolinea. Le fermate e il capolinea sono caratterizzate dalla loro posizione geografica, ad esempio in coordinate GPS. Un autobus è caratterizzato dalla linea che percorre, dalla posizone aggiornata in tempo reale, e dal fatto di essere completo oppure no. Ogni volta che l'autobus aggiorna la propria posizione, il sistema calcola il tempo stimato di arrivo a tutte le fermate della linea che sta percorrendo ed informa gli utenti del servizio tramite il loro smartphone, oltre che visualizzare i tempi di attesa presso pannelli informativi alle fermate stesse.

Si disegni un diagramma UML a scelta che illustra le relazioni tra le varie entita' coinvolte. Nel caso in cui si ritenga di voler applicare un determinato pattern, si indichi quale e quali entita' dell'applicazione rivestono quale ruolo nel pattern.

Soluzione:

Si tratta di un'applicazione del pattern Observer.

Esercizio 5

Si consideri il seguente frammento di codice Java:

```
public class ContoCorrente {
  private double saldo;
  private double numero;

  public synchronized deposita(double quantita){
     saldo += quantita;
  }

  public synchronized void ritira(double quantita){
     saldo -= quantita;
  }


  public synchronized void giroConto(ContoCorrente destinazione, double quantita){
      this.ritira(quantita);
      destinazione.deposita(quantita);
  }

}
```

Il frammento funziona correttamente in caso di accesso concorrente? In caso positivo, motivare la risposta. In caso negativo, mostrare un esempio del problema individuato e indicare come correggerlo.

Soluzione:

Il programma puo' andare in deadlock se, dati due conti correnti c1 e c2, un thread esegue ad es. `c1.giroConto(c2,100);` e un altro thread esegue `c2.giroConto(c1,50);`.

Infatti, il primo thread deve bloccare c1, l'altro c2. Pero', l'esecuzione di `c1.giroConto(c2,100)` ha bisogno di prendere il lock anche su c2 per continuare; analogamente, l'esecuzione di `c2.giroConto(c1,50);` ha bisogno del lock su c1. I due thread restano quindi entrambi in attesa perenne che l'altro liberi il lock.

Si puo' risolvere il problema molto semplicemente eliminando la dichiarazione `synchronized` dalla dichiarazione di `giroConto()`.

Si noti che il programma consente al saldo di diventare negativo (il metodo `ritira` non ha alcun vincolo): questo fatto puo' essere accettabile oppure no, a seconda della specifica del sistema, ma non è in questo caso un problema di concorrenza.